Yasmina's HireGuard pitch brief

Use this as a plain-language study note alongside the timed demo in demo_script.md. It reflects the current project notes on 1 November 2026. Re-check the Evaluation page before quoting any results.

The project in one sentence

HireGuard is an AI-assisted HR workflow platform: AI reads and classifies documents, ordinary code checks evidence and applies rules, and people approve decisions and actions.

The opening to memorize

> HR teams spend too much time sorting CVs, handling candidate emails, and answering repeated employee questions. HireGuard helps organize that work. Its AI reads CVs and classifies emails, while the software checks evidence and prepares suggested next steps. A person remains responsible for decisions about people, and nothing important is sent or committed without approval. We built it around one principle: **the AI reads, code decides, humans approve.**

This opening describes the product's purpose and design, not a particular screen, statistic, or feature still being refined.

How the system works

There are two modules on one shared trust and approval foundation:

- **Recruit / HireGuard:** CV review and ranking, inbox triage, interview

scheduling, follow-ups, and offer workflows.

- **Team / HR Nexus:** handbook answers, team skills coverage, and HR process

metrics. It is intended for team-level insights, not individual employee scoring.

The backend is a FastAPI application; the interface is a React application. The main workflow is:

1. **Prepare the input:** code checks a CV and removes or reports hidden text.
2. **Read:** the model extracts structured facts from a CV or classifies an

email.

1. **Check:** code validates CV claims against visible evidence, calculates

rubric scores, and applies routing and risk rules.

1. **Review:** HR sees the evidence and proposed action, then approves or

rejects it.

1. **Act and record:** only the executor performs approved actions; actions

and decisions are audited.

In the email flow, the model supplies limited fields such as language, intent, and urgency. Code checks sender authentication and candidate identity and decides whether the message is routine, suspicious, or needs a person. The Answer Desk is different: it searches the handbook and returns exact passages or routes a question to HR; it does not use a generative model to write answers.

The AI model and what “agents” means here

- The configured model is **Qwen3-8B**, served by **Ollama**. Local Ollama is

the default; remote hosting is optional.

- The model has bounded reading tasks and returns typed JSON. In the CV flow,

it does not set the score or make a hiring decision.

- The model calls do not have tools, persistent memory, or authority to send

messages, book interviews, or change records.

- Backend code coordinates the stages and enforces the approval process.

So, describe this accurately as a **guarded AI-assisted workflow with orchestration**. It is not a group of autonomous AI agents talking to one another. If asked, explain that the model is deliberately limited: reliable validation and human control are more important here than giving it free reign.

What is distinctive

- **Separate reading from acting:** untrusted CV and email text is read in a

constrained pipeline; the executor that acts does not make decisions from that text.

- **Evidence-backed suggestions:** skills and other CV claims must be grounded

in visible text before scoring. The score is a proposal, with justifications.

- **Human approval enforced by the backend:** an unapproved action is refused;

offers can require a second approver.

- **Safety without pretending there is no risk:** hidden content is surfaced

for review, and stability/fairness tools help identify cases that deserve attention.

- **Privacy-conscious Team insights:** reporting focuses on declared skills,

process stages, and handbook topics rather than individual employee scores.

- **Practical offer-fit check:** compare a candidate's declared conditions with

the published offer terms instead of predicting acceptance using sensitive proxies.

Limitations to understand and explain

- **Model fairness is not solved.** The pitch notes report that 26 of 90 twin

CV variants changed score in the measured model run, before identity masking. Masking was added afterward; the re-measurement is pending. Say this plainly if asked about bias.

- **Model quality evidence is incomplete.** The pitch notes say extraction

quality was measured on only two CVs; that is not enough to claim broad accuracy.

- **Separate model results from code-path results.** Ground-truth/oracle runs

test the pipeline and deterministic code, not the model. The Evaluation screen labels the run mode; use it when quoting a metric.

- **This is a demo, not a production deployment.** Demo email sending and

calendar booking use mock services; SQLite is used in the demo. Production integrations and deployment need their own validation.

- **Demo data is fictional.** The model runs locally by default, but a cloud

GPU has also been used for the hackathon. Do not promise that data always stays on one machine.

Easy answers for likely questions

“Does the AI rank or hire people?” “The model extracts facts. Code applies the job rubric and shows a proposed score with evidence. A person decides; the AI does not hire anyone.”

“Are these autonomous agents?” “There is orchestration across workflow stages, but not a team of autonomous agents. We use one constrained model for reading and classification, with deterministic code and human approval around it.”

“Can a malicious CV manipulate the result?” “We test for hidden text and prompt attacks, but we do not claim perfect protection. The model's output is checked against visible evidence, and we report measured failures and known limits.”

“What is the key idea?” “The system can take paperwork off HR's hands without handing over judgment about people.”

Pitch discipline

- Never say **“the AI decided,” “the AI hired,”** or **“attacks never work.”**
- For statistics, read the current **Security Center** → **Evaluation** result and

name whether it is a model run or code-path run.

- If the demo uses offline/ground-truth data, say so; do not present it as a

live model result.

- Keep the story simple: **problem → bounded AI → code checks → human approval

→ useful action**.